Security

Guarantees by design

- Keys never in C#, Angular, Git, logs, or API responses (pointers + `IConfiguration` at startup; `/info` returns booleans + config paths only).
- Browser calls only YOUR domain (`/api/ai/*`); upstream hosts are unreachable from the frontend. Pasting keys into any UI is refused by design — set them server-side (User Secrets / env / Key Vault / AWS Secrets Manager).
- Safe errors: envelope codes (`VALIDATION_ERROR`, `PROVIDER_NOT_CONFIGURED`, `*_UNAVAILABLE`, `INTERNAL_ERROR`) with `traceId`; upstream bodies truncated to 300 chars; no stack traces or internals leak.
- Structured logs carry provider/operation/duration/outcome — never prompts, keys, or `Authorization` headers. Prompt-injection note: system instructions stay server-side (`ChatDefaults:SystemPrompt`); treat user input as data, never as instructions; render markdown as sanitized HTML in your UI.

Settings panel contract

- **Gate**: `POST /settings/login` (password → 8h token) — wrong password 401; 5 failures/IP → lockout (15 min, then 60 s retry). Every other settings route requires `X-Settings-Token` (or `Bearer`) → else 401 `UNAUTHORIZED`.
- **PBKDF2** password hash persisted in the store; `Settings:Password` only seeds the first run (`passwordSource: config` → `stored` after a change).
- **Raw keys are write-only**: they go in on authenticated HTTPS and come back only as `apiKeyMasked` (`sk-a…1234`) + `hasKey`. Blank/omitted key on any save keeps the stored secret — masked values can never round-trip into a real key. Voice keys behave the same (`sttKeyMasked`/`ttsKeyMasked`).
- **Disk**: `AllowPersistKeys=true` (default) stores keys in `ai-suite-settings/settings.json` — keep that directory out of Git and backups you don't control; `false` keeps keys memory-only (restart re-seeds).
- **Logs**: store operations log outcomes only — no keys, no passwords, no prompts. Key-test failures store a `SecretFree` upstream message.
- **Client**: `AiSettingsService` holds the token in `sessionStorage` only, clears it on any 401 and emits `unauthorized$` so your UI re-opens the gate.

Mandatory host checklist

```
builder.Services.AddCors(o =&gt; o.AddPolicy("FrontendOnly", p =&gt; p
    .WithOrigins("https://app.mycompany.com") // exact origins — never AllowAnyOrigin + AllowCredentials
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials()));
// + AddAuthentication (JWT/OIDC) → AiSuite:Security:RequireAuthorization = true
// + AiSuite:RateLimit:Enabled = true → app.UseRateLimiter()
// + guard POST /api/ai/selection with an admin role/policy

provideHttpClient(withInterceptors([aiSuiteAuthInterceptor(() =&gt; tokenService.getToken())]))
```

Prefer Managed Identity (`DefaultAzureCredential`) so there is no key to leak; rotate any key that ever touches logs, chat history, or the browser. Pre-commit: `appsettings.*` must contain only `ApiKeyConfigKey` pointers (root `.gitignore` covers secrets, `.env`, `bin/`, `obj/`, `node_modules/`).